⚡ Lab 6: Configure High Availability

AWS Technical Essentials

Load Balancing, Auto Scaling & Self-Healing

🏷️ Resource Names for This Lab

Replace <ID> with your assigned student ID (for example user7).

Resource	Name
Application Load Balancer	employee-directory-alb-<ID>
ALB security group	employee-directory-alb-sg-<ID>
Target group	employee-directory-tg-<ID>
Launch template	employee-directory-lt-<ID>
Auto Scaling group	employee-directory-asg-<ID>
Reused from earlier labs	employee-directory-<ID>-vpc, employee-directory-app-sg-<ID>, employee-dir-key-<ID>

> ⚠️ **32-character limit:** AWS caps load balancer and target group names at 32 characters. Keep your ID to six characters or fewer.

📋 Lab Overview

🎯 Objectives

- Create an Application Load Balancer
- Create a Launch Template
- Configure an Auto Scaling Group
- Test self-healing
- View CloudWatch metrics

📝 Lab Details

- **Duration:** ~50 minutes
- **Capstone:** Lab 6 of 6

✅ Prerequisites

> **Before starting:** Verify these resources from previous labs exist.

- **VPC:** employee-directory-<ID>-vpc with 2 public subnets in different AZs
- **Security Group:** employee-directory-app-sg-<ID>
- **Key Pair:** employee-dir-key-<ID>

> 💡 **Tip:** Terminate any running EC2 instances from earlier labs to stay within limits.

🌐 Enable Public IPs on the Subnets

The VPC wizard leaves **auto-assign public IPv4** off. Auto Scaling instances inherit the subnet setting, and without a public IP they cannot reach the internet to install Apache.

1. Navigate to **VPC** → **Subnets**
2. Filter by your ID and select the first **public** subnet
3. Click **Actions** → **Edit subnet settings**
4. Check **Enable auto-assign public IPv4 address**, then **Save**
5. Repeat for the second public subnet

> ✅ **Expected:** Both public subnets show **Auto-assign public IPv4: Yes**.

🔧 Create the Application Load Balancer

1. Navigate to **EC2** → **Load Balancers** (left nav)
2. Click **Create load balancer** → **Application Load Balancer** → **Create**

Setting	Value
Name	`employee-directory-alb-<ID>`
Scheme	**Internet-facing**
VPC	**employee-directory-<ID>-vpc**
Availability Zones	Check **both** public subnets

Create a **new security group** for the ALB:

- Name: `employee-directory-alb-sg-<ID>`
- Inbound rule: **HTTP / 80** from `0.0.0.0/0`

🔧 Create Target Group and Finish ALB

Under **Listeners and routing**, click **Create target group**:

Setting	Value
Target type	**Instances**
Name	`employee-directory-tg-<ID>`
Protocol / Port	HTTP / 80
VPC	employee-directory-<ID>-vpc
Health check path	`/`

1. Click **Next**, skip registering targets, click **Create target group**
2. Return to the ALB tab, **refresh**, select **employee-directory-tg-<ID>**
3. Click **Create load balancer**

> ✅ **Expected:** ALB created. Status will show **Provisioning** for a few minutes.

📄 Create a Launch Template

1. Navigate to **EC2 → Launch Templates**
2. Click **Create launch template**

Setting	Value
Name	`employee-directory-lt-<ID>`
AMI	**Amazon Linux 2023**
Instance type	**t2.micro**
Key pair	**employee-dir-key-<ID>**
Security group	**employee-directory-app-sg-<ID>**

Under **Advanced details**, scroll to **User data** and paste the script on the next slide.

📄 Launch Template — User Data

Paste into the **User data** field:

```
#!/bin/bash -ex
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd
EC2_INSTANCE_ID=$(ec2-metadata -i | cut -d" " -f2)
EC2_AZ=$(ec2-metadata -z | cut -d" " -f2)
echo "<!DOCTYPE html>
<html><head><title>Employee Directory</title>
<style>body{font-family:Arial,sans-serif;max-width:800px;margin:50px auto;padding:20px}
h1{color:#1a365d}table{border-collapse:collapse;width:100%}th,td{border:1px solid #ddd;padding:12px;text-align:left}
th{background:#2b6cb0;color:white}tr:nth-child(even){background:#f7fafc}</style></head>
<body><h1>Employee Directory</h1>
<p>Instance: $EC2_INSTANCE_ID | AZ: $EC2_AZ</p>
<table><tr><th>Name</th><th>Department</th><th>Location</th></tr>
<tr><td>John Smith</td><td>Engineering</td><td>Seattle</td></tr>
<tr><td>Jane Doe</td><td>Marketing</td><td>New York</td></tr>
<tr><td>Bob Johnson</td><td>Sales</td><td>Austin</td></tr></table>
</body></html>" > /var/www/html/index.html
```

1. Click **Create launch template**

📊 Create Auto Scaling Group

1. Navigate to **EC2 → Auto Scaling Groups**
2. Click **Create Auto Scaling group**
3. Name: `employee-directory-asg-<ID>`
4. Launch template: **employee-directory-lt-<ID>**
5. Click **Next**
6. VPC: **employee-directory-<ID>-vpc**
7. Subnets: Select **both** public subnets
8. Click **Next**

📊 ASG — Attach to ALB

1. Select **Attach to an existing load balancer**
2. Choose **Choose from your load balancer target groups**
3. Select **employee-directory-tg-<ID>**
4. Health check type: Check **Turn on Elastic Load Balancing health checks**
5. Click **Next**

Setting	Value
Desired capacity	**2**
Min	**2**
Max	**4**

6. Click **Next** through remaining pages, then **Create Auto Scaling group**

🌐 Verify — Test the ALB

1. Go to **EC2** → **Instances** — confirm 2 new instances are **Running**
2. Go to **Load Balancers** → select your ALB
3. Copy the **DNS name**
4. Open in a browser: `http://<ALB_DNS_NAME>`
5. Refresh the page several times

> ✅ **Expected:** The Employee Directory page loads. Refreshing shows different **Instance IDs** and **AZs** — the ALB is distributing traffic.

🔨 Test Self-Healing

1. Go to **EC2 → Instances**
2. Select one of the ASG instances
3. Click **Instance state → Terminate instance**
4. Watch the **Auto Scaling Groups** activity tab

> ✅ **Expected:** Within 1-2 minutes, Auto Scaling launches a **replacement instance** to maintain the desired count of 2. The ALB continues serving traffic through the remaining healthy instance.

📊 View CloudWatch Metrics

1. Go to **EC2** → select an instance → **Monitoring** tab
2. Review: CPU utilization, network in/out, status checks
3. Go to **Load Balancers** → select ALB → **Monitoring** tab
4. Review: request count, healthy host count, response time

> 💡 **Key metrics:** The ALB **Healthy Host Count** dropped to 1 when you terminated an instance and returned to 2 when the replacement launched.

🗑️ Clean Up — Step 1

> ⚠️ **Delete in this order** to avoid dependency errors:

1. **Auto Scaling Group:** EC2 → Auto Scaling Groups → select → Delete (set desired/min to 0, then delete)
2. **Wait** for all ASG instances to terminate
3. **Load Balancer:** EC2 → Load Balancers → select → Delete
4. **Target Group:** EC2 → Target Groups → select → Delete

🗑️ Clean Up — Step 2

5. **Launch Template:** EC2 → Launch Templates → select → Delete
6. **Security Groups:** Delete `employee-directory-alb-sg-<ID>` and `employee-directory-app-sg-<ID>`
7. **VPC:** VPC → Your VPCs → select employee-directory-<ID>-vpc → Delete (this removes subnets, route tables, and IGW)
8. **Key Pair:** EC2 → Key Pairs → delete `employee-dir-key-<ID>`
9. **Any remaining EC2 instances** from earlier labs → Terminate

> ⚠️ If VPC deletion fails, check for remaining ENIs or instances. All dependencies must be removed first.

🎉 Lab 6 Complete!

High Availability Architecture Deployed

🎯 Key Takeaways

🛠️ What You Built

- Application Load Balancer across 2 AZs
- Launch Template with bootstrap script
- Auto Scaling Group with self-healing
- CloudWatch monitoring

📚 Key Concepts

- **ALB** distributes traffic across AZs
- **ASG** maintains desired instance count
- **Self-healing** replaces failed instances
- **CloudWatch** provides monitoring

🏆 All Labs Complete!

AWS Technical Essentials

You have built and deployed across IAM, EC2, VPC, S3, DynamoDB, ALB, and Auto Scaling



Questions?

Final questions before we wrap up the course